

1. regex
2. - at least 3
- no digits between vowels
3. sed
4. awk
5. n

$$\begin{array}{r} 10 \\ 61- \\ 25 \\ \hline 36 \end{array}$$

$$\begin{array}{r} 48- \\ 32 \\ \hline 61 \end{array}$$

$$\begin{array}{r} 10 \\ 62- \\ 37 \\ \hline 25 \end{array} \quad 6'$$

✓ 6. only if the third file is not one of the two.

7. - p → exit(0)

↳ c1

P
↳ c1

8. -

9. -

10. -

~ 11. -

12. -

↔ rlock - wlock

15. sem wait ↔ ~~rlock - wlock~~ rlock - rdlock

rlock ↔ bin.sem

16

17 ✓

18

19

20

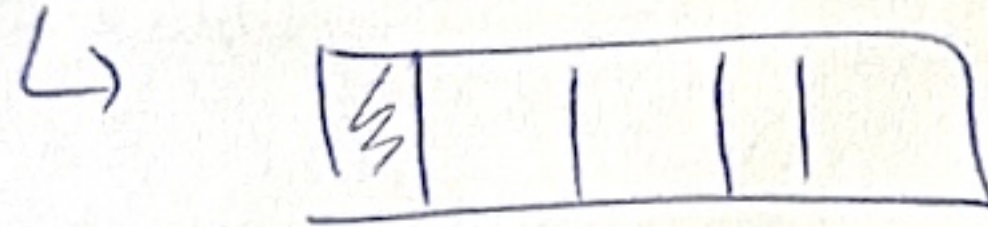
EXAM 914
2026

Theory

1) INODE (index node)

↳ metadata $\leftarrow$ $\begin{matrix} \text{length} \\ \text{perm.} \\ \text{loc.} \end{matrix}$

2) External fragmentation



3) Deadlock.

T₁ - holds L₁

~~T₂ - holds~~

- waits for L₂

holds

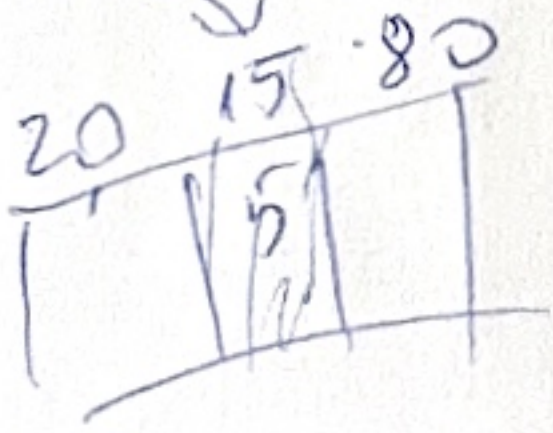
T₂ - holds L₂

- waits for L₁

H H N C

H N H C

- ! ALL
- 1) mutual exclusion = threads claim exclusive control over their resources.
 - 2) hold-and-wait = threads hold resources while waiting for other resources.
 - 3) no preemption = resources cannot be forcibly removed from threads that are holding them.
 - 4) circular wait = $\exists$ a circular chain of threads s.t. each thread holds one or more resources that are being requested by the next thread in the chain.



4) Prevent circular wait

$\Rightarrow$ provide a total ordering on lock acquisition (lock in order)

5) Prevent hold-and-wait

$\Rightarrow$ acquire all locks at once.

```
pthread_mutex_lock (prevention);
pthread_mutex_lock (L1);
    - - - - - (L2)
pthread_mutex_unlock (prevention)
```

7) Prevent mutual exclusion
 $\Rightarrow$ use lock-free / wait-free algs.

try atomic updates
until success

no mutex $\Rightarrow$ no deadlock, but livelock

! 6) Prevent no-preemption
 $\Rightarrow$ trylock + backoff.

T₁ owns L₁ and tries to get L₂ $\Rightarrow$ deadlock.
T₂ owns L₂ and tries to get L₁.

pthread_mutex_trylock(), then release already-held locks if acquisition fails.

Ex: T₁ gets L₁ but can't get L₂ $\rightarrow$ release L₁, then retry later.

lock(L₁) $\rightarrow$ if trylock(L₂) fails $\rightarrow$ { unlock(L₁)
retry